淺度機器學習分類器比較 -- 使用資料 allFaces，並將其設定為ds1

學號：411079060

姓名：石欽鈺

完成日期：2023/5/12

作品目標： 在這個數據龐大的時代，多變量、大數據、即時反應已是很常見的場景。此時，機器學習是解決這些需求的工具，而我們想知道不同的分類器在不同資料面前的性能，於是進行 評比實驗 來探討機器學習中常見的分類方法，包括 多元羅吉斯回歸（MLR）、支援向量機（SVM）、以及神經網路（Neural Network），同時我們也想知道用原始資料與主成分資料於分類時的差異，無論是準確度的變化還是整體時間耗費多寡。因此，我們還會將資料分成原始資料與主成分資料，除此之外，部分資料還有做看看降噪處理，並去觀察他的性能：

1. 介紹並認識三種常見的統計分類方法：多元羅吉斯回歸、支援向量機、神經網路。
2. 評比實驗：
   - 使用三種不同的資料集，包括大小、變數多寡、資料型態等不同特徵的資料，來測試三種分類器的準確度。
   - 準確度的優化: 包括但不限於參數的調整（如正規化、gamma、degree等）、計算方式的選擇（如演算法、激活函數等）、以及資料預處理（如標準化、PCA / SVD、降噪等）。
3. 針對不同資料：原始 & 0.95比例的主成份，找出最佳準確度組合，並去比較後哪種分類器在影像分類任務中表現最佳，不同資料型態在分類時會產生什麼樣的差異，並給出相應的建議和結論。 </font>

In [ ]:

```
import scipy.io

# Load data from allFaces.mat
data = scipy.io.loadmat('allFaces.mat')
# Check keys in the dictionary
print(data.keys())
```

```
dict_keys(['__header__', '__version__', '__globals__', 'faces', 'n', 'm', 'sub', 'nfaces', 'person'])
```

- 人臉資料的輸入需非常小心地去對每個變數的內容物為何？有多少？
- 善用初始值設定和空的list來存放資料
- x, y的資料長相非常重要！！

In [ ]:

```
import scipy.io
import numpy as np

# Load data from allFaces.mat
data = scipy.io.loadmat('allFaces.mat')

# Extract features (X) and labels (y)
X = data['faces']
X = X.T
z = data['nfaces'].flatten()  # Assuming 'person' contains the labels

# Initialize variables
n = 0
lst = []

# Iterate through elements of z
for i in z:
    for _ in range(i):
        lst.append(n)
    n += 1

# Convert list to NumPy array
y = np.array(lst)

# Check the shape of X and y
print("Shape of X:", X.shape)
print("Shape of y:", y.shape)

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Split data into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)

# Standardize data
scaler = StandardScaler()
X_train_ = scaler.fit_transform(X_train)
X_test_ = scaler.transform(X_test)
```

```
Shape of X: (2410, 32256)
Shape of y: (2410,)
```

- 問題與討論
  - 要先確認要使用dict中的哪些key，以確保接下來的 x, y 值設定能成功。原先沒有去檢查，直接使用如 ds0, ds1 的程式碼，導致不成功。
  - 輸出及輸入的數量要去終端機的筆記本中檢查一下變數資訊，原先忘了轉至矩陣，導致數量無法對應。
  - 38 個人臉中，每個人約由60多張，那不能直接用“38”這個數字去跑，要先將每個人乘上他們的張數，讓最終能有 2410 筆，輸出輸入才能相對應。

ds2 -- lr

In [ ]:

```python
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Split data into training and testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)

# Standardize data
scaler = StandardScaler()
X_train_ = scaler.fit_transform(X_train)
X_test_ = scaler.transform(X_test)

# Logistic Regression model
opts = dict(tol=1e-5, max_iter=int(1e5), verbose=0)
solver = 'liblinear'
clf_original = LogisticRegression(solver=solver, **opts)
clf_original.fit(X_train_, y_train)
y_pred = clf_original.predict(X_test_)

# Report accuracy on test data
print(f"Accuracy using clf_original.score: {clf_original.score(X_test_, y_test):.2%}\n")
```

```
Accuracy using clf_original.score: 98.89%
```

ds2 -- lr -- pca

In [ ]:

```python
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# PCA transformation
pca = PCA(n_components=0.95).fit(X_train_)
Z1_train = pca.transform(X_train_)
Z1_test = pca.transform(X_test_)

opts = dict(tol=1e-6, max_iter=int(1e6), verbose=0)
# solver = 'lbfgs'  # 'lbfgs' is the default
# solver = 'liblinear'
solver = 'newton-cg'

# Logistic Regression with PCA-transformed data
clf_PCA = LogisticRegression(solver=solver, **opts)
clf_PCA.fit(Z1_train, y_train)
y_pred = clf_PCA.predict(Z1_test)

# Report accuracy on test data
print(f"Accuracy using clf_PCA.score: {clf_PCA.score(Z1_test, y_test):.2%}\n")
```

```
Accuracy using clf_PCA.score: 94.05%
```

ds2 -- svm

In [ ]:

```python
from sklearn.svm import SVC, LinearSVC
from sklearn.metrics import classification_report

C = 0.1  # SVM regularization parameter
clf_svm = SVC(kernel="linear", **opts)
# clf_svm = SVC(kernel="rbf", gamma=0.3, **opts)
# clf_svm = SVC(kernel="poly", degree=3, gamma="auto", **opts)
# clf_svm = LinearSVC(**opts)  # one vs the rest

clf_svm.fit(X_train_, y_train)
predictions = clf_svm.predict(X_test_)
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.94      1.00      0.97        15
```

```
           1       0.83      0.95      0.89        21
           2       0.80      1.00      0.89        16
           3       0.89      1.00      0.94        25
           4       0.96      1.00      0.98        26
           5       0.94      1.00      0.97        16
           6       0.96      0.92      0.94        25
           7       0.77      1.00      0.87        20
           8       1.00      0.96      0.98        24
           9       0.81      0.85      0.83        20
          10       1.00      0.95      0.97        19
          11       1.00      1.00      1.00        11
          12       1.00      0.89      0.94        18
          13       1.00      1.00      1.00        18
          14       0.95      0.95      0.95        19
          15       0.81      0.93      0.87        14
          16       1.00      1.00      1.00        17
          17       1.00      1.00      1.00        20
          18       0.88      0.88      0.88        16
          19       0.90      0.95      0.93        20
          20       1.00      0.87      0.93        23
          21       1.00      0.94      0.97        18
          22       1.00      0.88      0.94        26
          23       0.95      1.00      0.98        20
          24       0.95      0.95      0.95        19
          25       1.00      1.00      1.00        17
          26       1.00      0.94      0.97        16
          27       1.00      0.83      0.91        24
          28       0.96      0.92      0.94        26
          29       0.92      1.00      0.96        22
          30       0.94      0.94      0.94        18
          31       0.73      0.85      0.79        13
          32       0.84      0.89      0.86        18
          33       1.00      0.88      0.94        17
          34       1.00      0.72      0.84        18
          35       1.00      0.92      0.96        13
          36       0.95      0.95      0.95        19
          37       1.00      0.81      0.90        16

    accuracy                           0.93       723
   macro avg       0.94      0.93      0.93       723
weighted avg       0.94      0.93      0.94       723
```

ds2 -- svm -- PCA

In [ ]:

```python
from sklearn.svm import SVC, LinearSVC
from sklearn.metrics import classification_report

C = 0.1  # SVM regularization parameter
opts = dict(C=C, tol=1e-6, max_iter=int(1e6))

# Choose one of the following classifiers by uncommenting the appropriate line
clf_svm = SVC(kernel="linear", **opts)
# clf_svm = SVC(kernel="rbf", gamma=0.2, **opts)
# clf_svm = SVC(kernel="poly", degree=1, gamma="auto", **opts)
# clf_svm = LinearSVC(**opts)  # one vs the rest

# Train the classifier
clf_svm.fit(Z1_train, y_train)

# Make predictions
predictions = clf_svm.predict(Z1_test)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       0.82      0.93      0.88        15
           1       0.76      0.90      0.83        21
           2       0.88      0.94      0.91        16
           3       0.89      0.96      0.92        25
           4       1.00      1.00      1.00        26
           5       0.88      0.94      0.91        16
           6       0.96      0.88      0.92        25
           7       0.83      1.00      0.91        20
           8       1.00      0.96      0.98        24
           9       0.81      0.85      0.83        20
          10       1.00      0.84      0.91        19
          11       1.00      1.00      1.00        11
          12       1.00      0.83      0.91        18
          13       1.00      1.00      1.00        18
          14       0.86      1.00      0.93        19
          15       0.72      0.93      0.81        14
          16       0.94      1.00      0.97        17
          17       0.91      1.00      0.95        20
          18       0.88      0.94      0.91        16
          19       1.00      1.00      1.00        20
          20       1.00      0.91      0.95        23
          21       0.89      0.89      0.89        18
```

```
          22       1.00      0.85      0.92        26
          23       1.00      1.00      1.00        20
          24       0.90      0.95      0.92        19
          25       1.00      1.00      1.00        17
          26       0.94      1.00      0.97        16
          27       0.95      0.83      0.89        24
          28       0.92      0.92      0.92        26
          29       0.88      1.00      0.94        22
          30       1.00      0.94      0.97        18
          31       0.80      0.92      0.86        13
          32       1.00      0.83      0.91        18
          33       0.94      0.88      0.91        17
          34       1.00      0.72      0.84        18
          35       1.00      0.92      0.96        13
          36       1.00      0.95      0.97        19
          37       1.00      0.81      0.90        16

    accuracy                           0.93       723
   macro avg       0.93      0.93      0.93       723
weighted avg       0.93      0.93      0.93       723
```

ds2 -- nn

In [ ]:

```python
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

# Set the parameters for the MLPClassifier
hidden_layers = (120,)  # one hidden layer with 30 units
activation = 'logistic'
opts = dict(hidden_layer_sizes=hidden_layers, verbose=False, activation=activation, tol=1e-6, max_iter=1000)
solver = 'lbfgs'  # default solver

# Create the MLPClassifier
clf_MLP = MLPClassifier(solver=solver, **opts)

# Train the classifier
clf_MLP.fit(X_train_, y_train)

# Make predictions
predictions = clf_MLP.predict(X_test_)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       1.00      0.93      0.97        15
           1       0.94      0.81      0.87        21
           2       0.76      0.81      0.79        16
           3       0.89      0.96      0.92        25
           4       0.96      1.00      0.98        26
           5       0.89      1.00      0.94        16
           6       0.95      0.80      0.87        25
           7       0.91      1.00      0.95        20
           8       0.91      0.83      0.87        24
           9       0.89      0.80      0.84        20
          10       0.83      0.79      0.81        19
          11       0.83      0.91      0.87        11
          12       1.00      0.94      0.97        18
          13       0.95      1.00      0.97        18
          14       0.90      1.00      0.95        19
          15       0.86      0.86      0.86        14
          16       1.00      1.00      1.00        17
          17       1.00      1.00      1.00        20
          18       0.94      0.94      0.94        16
          19       0.95      0.95      0.95        20
          20       0.85      0.96      0.90        23
          21       0.88      0.83      0.86        18
          22       1.00      0.92      0.96        26
          23       1.00      0.85      0.92        20
          24       0.95      0.95      0.95        19
          25       0.94      1.00      0.97        17
          26       1.00      1.00      1.00        16
          27       1.00      0.79      0.88        24
          28       0.86      0.92      0.89        26
          29       1.00      0.95      0.98        22
          30       1.00      1.00      1.00        18
          31       1.00      1.00      1.00        13
          32       0.95      1.00      0.97        18
          33       0.85      1.00      0.92        17
          34       0.94      0.83      0.88        18
          35       0.86      0.92      0.89        13
          36       0.86      0.95      0.90        19
          37       0.84      1.00      0.91        16

    accuracy                           0.92       723
   macro avg       0.92      0.93      0.92       723
weighted avg       0.93      0.92      0.92       723
```

In [ ]:

```python
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

# Set the parameters for the MLPClassifier
hidden_layers = (120,)  # one hidden layer with 30 units
activation = 'logistic'
opts = dict(hidden_layer_sizes=hidden_layers, verbose=False, activation=activation, tol=1e-6, max_iter=1000,
       )
solver = 'sgd'  # default solver

# Create the MLPClassifier
clf_MLP = MLPClassifier(solver=solver, **opts)

# Train the classifier
clf_MLP.fit(Z1_train, y_train)

# Make predictions
predictions = clf_MLP.predict(Z1_test)

# Print classification report
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       1.00      0.93      0.97        15
           1       0.89      0.76      0.82        21
           2       1.00      0.94      0.97        16
           3       0.86      0.96      0.91        25
           4       1.00      1.00      1.00        26
           5       1.00      1.00      1.00        16
           6       1.00      0.80      0.89        25
           7       0.95      1.00      0.98        20
           8       0.96      0.92      0.94        24
           9       1.00      0.95      0.97        20
          10       0.94      0.89      0.92        19
          11       1.00      1.00      1.00        11
          12       0.95      1.00      0.97        18
          13       0.95      1.00      0.97        18
          14       0.90      1.00      0.95        19
          15       0.87      0.93      0.90        14
          16       0.85      1.00      0.92        17
          17       0.95      1.00      0.98        20
          18       0.88      0.94      0.91        16
          19       1.00      1.00      1.00        20
          20       1.00      0.91      0.95        23
          21       0.94      0.89      0.91        18
          22       1.00      0.92      0.96        26
          23       1.00      0.90      0.95        20
          24       1.00      1.00      1.00        19
          25       1.00      1.00      1.00        17
          26       1.00      1.00      1.00        16
          27       1.00      0.92      0.96        24
          28       0.92      0.92      0.92        26
          29       1.00      1.00      1.00        22
          30       0.86      1.00      0.92        18
          31       0.86      0.92      0.89        13
          32       0.95      1.00      0.97        18
          33       0.89      1.00      0.94        17
          34       0.83      0.83      0.83        18
          35       1.00      0.92      0.96        13
          36       0.86      0.95      0.90        19
          37       1.00      1.00      1.00        16

    accuracy                           0.95       723
   macro avg       0.95      0.95      0.95       723
weighted avg       0.95      0.95      0.95       723
```

```
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/site-packages/sklearn/neural_network/_multila
yer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the opti
mization hasn't converged yet.
  warnings.warn(
```

In [ ]:

```python
# from sklearn.metrics import confusion_matrix
# import matplotlib.pyplot as plt
# import numpy as np

# # 假設 clf_MLP 是你的模型,X_test 和 y_test 是測試數據
# y_pred = clf_MLP.predict(X_test)

# # 計算混淆矩陣
# cm = confusion_matrix(y_test, y_pred)
# cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

# # 繪製混淆矩陣
# plt.imshow(cm_normalized, interpolation='nearest', cmap=plt.cm.Blues)
# plt.title('Normalized Confusion Matrix')
```

```
# plt.colorbar()

# # 添加標籤
# classes = ['Class 0', 'Class 1', 'Class 2']  # 替換為你的類別標籤
# tick_marks = np.arange(len(classes))
# plt.xticks(tick_marks, classes)
# plt.yticks(tick_marks, classes)

# # 添加數字
# thresh = cm_normalized.max() / 2.
# for i in range(cm_normalized.shape[0]):
#     for j in range(cm_normalized.shape[1]):
#         plt.text(j, i, format(cm_normalized[i, j], '.2f'),
#                  ha="center", va="center",
#                  color="white" if cm_normalized[i, j] > thresh else "black")

# plt.ylabel('True label')
# plt.xlabel('Predicted label')
# plt.tight_layout()
# plt.show()
```

In [ ]:

```
import scipy.io
import numpy as np

# Load data from allFaces.mat
data = scipy.io.loadmat('allFaces.mat')

# Extract features (X) and labels (y)
X = data['faces']
X = X.T
z = data['nfaces'].flatten()  # Assuming 'person' contains the labels

# Initialize variables
lst = []
n = 0

# Iterate through elements of z
for i in z:
    for _ in range(i):
        lst.append(n)
    n += 1

# Convert list to NumPy array
y = np.array(lst)
```

In [ ]:

```
# from sklearn.metrics import ConfusionMatrixDisplay  # 導入 ConfusionMatrixDisplay
# import numpy as np
# from sklearn.preprocessing import StandardScaler
# from sklearn.model_selection import train_test_split
# from sklearn.linear_model import LogisticRegression
# from sklearn.metrics import accuracy_score, classification_report
# import scipy.io


# # Split data into training and testing data
# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)

# # Standardize data
# scaler = StandardScaler()
# X_train_ = scaler.fit_transform(X_train)
# X_test_ = scaler.transform(X_test)

# from sklearn.neural_network import MLPClassifier
# from sklearn.metrics import classification_report

# # Set the parameters for the MLPClassifier
# hidden_layers = (30,)  # one hidden layer with 30 units
# activation = 'logistic'
# opts = dict(hidden_layer_sizes=hidden_layers, verbose=False, activation=activation, tol=1e-6, max_iter=1000)
# solver = 'adam'  # default solver

# # Create the MLPClassifier
# clf_MLP = MLPClassifier(solver=solver, **opts)

# # Train the classifier
# clf_MLP.fit(X_train, y_train)

# score = 100 * clf_MLP.score(X_test, y_test)
# title = 'Testing score = {:.2f}%'.format(score)

# # 使用 ConfusionMatrixDisplay 代替 plot_confusion_matrix
# disp = ConfusionMatrixDisplay(confusion_matrix=confusion_matrix(clf_MLP, X_test, y_test),
#                               display_labels=clf_MLP.classes_)  # 如果 clf_MLP 是多類別分類器，需要提供類別標籤
```

```
# disp.plot(xticks_rotation=45, cmap=plt.cm.Blues, ax=ax)
# ax.set_title(title)  # 設置標題

# plt.show()
```

In [ ]:

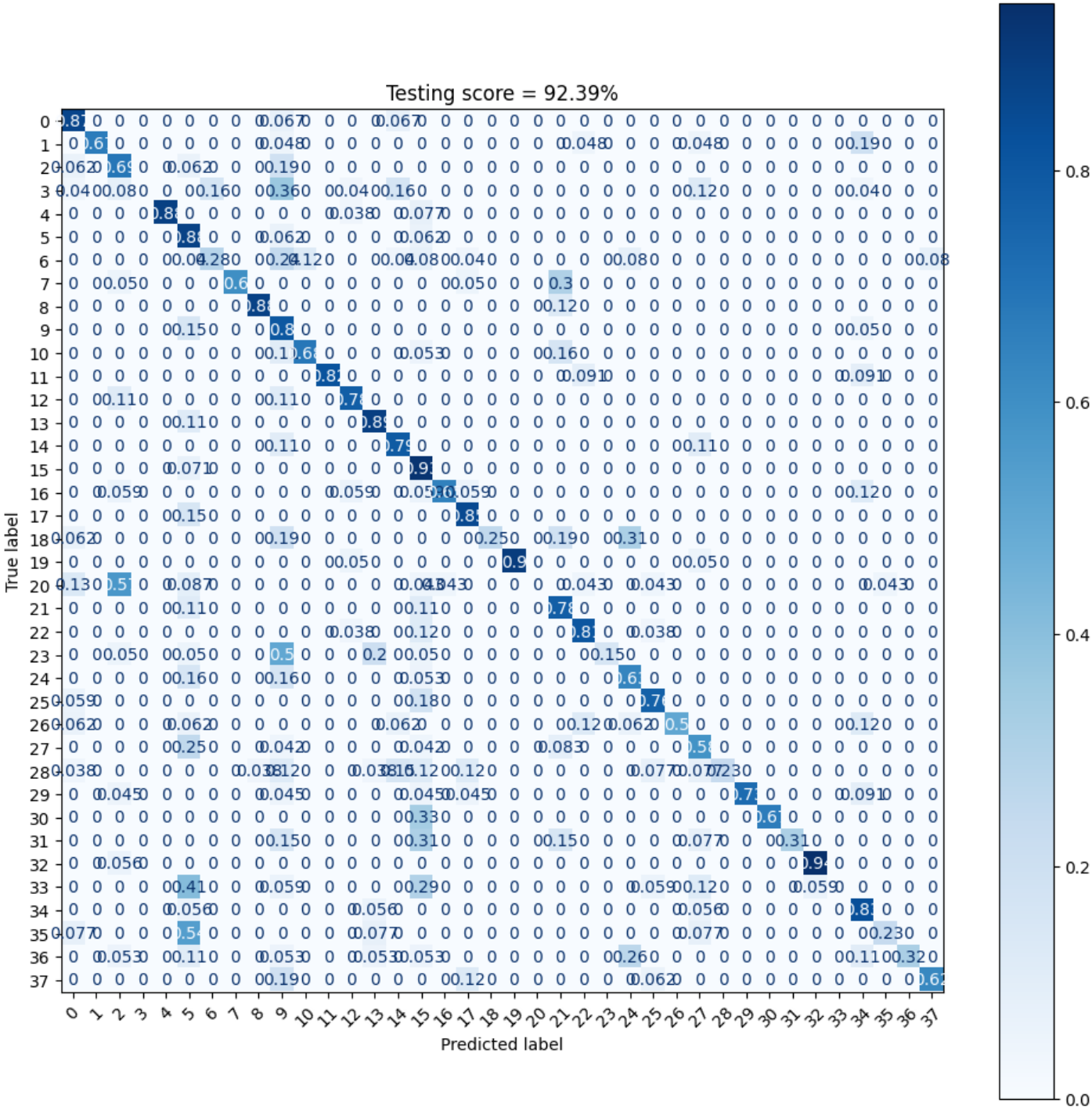
```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 1, figsize=(12,12))

score = 100 * clf_MLP.score(X_test_, y_test)
title = 'Testing score = {:.2f}%'.format(score)

disp = ConfusionMatrixDisplay.from_estimator(
    clf_MLP,
    X_test,
    y_test,
    xticks_rotation=45,
    # display_labels=class_names,
    cmap=plt.cm.Blues,
    normalize='true',
    ax=ax
)

disp.ax_.set_title(title)
plt.show()
```



- setting: 神經元網路分類器下分類標準化原始資料
  - 從上圖可發現很多類別都會有不少的比例被誤分到其他類，不過都是分散在各個其他類別中，所以真要取出比例最高的類別作為最後分類，基本上還是不會選錯。除了3.6.18.20.31.33.35.36這幾類被分錯到其他類的比例非常高，都大於0.7。可以去看看資料長相中這些部分發生什麼事

準確度成果

- 羅吉斯迴歸（Logistic Regression）
  - 標準化資料：98.99%
  - PCA資料：94.05%
- 支援向量機（Support Vector Machine, SVM）
  - 標準化資料：93%
  - denoise資料：92%
  - PCA資料：93%
- 神經網路（Multi-layer Perceptron, MLP）
  - 標準化資料：92%
  - PCA資料：95%

比較

1. 運用羅吉斯分類器，在沒有特別設定正規化參數下，採用liblinear演算法，產出98.89%的最佳準確度。不過其花了很多時間去跑（隨著我的電腦散熱程度從10,in~70min都有發生過），做了pca (取0.95比例)後，準確度下降近 5% ，這次改用newton-cg演算法，它雖然耗費了更多時間，但卻相對liblinear獲得更好的準確度。
2. 在svm分類器中，我皆選用linear演算法，並將正規化參數設為0.1。在最佳組合下，標準化和pca資料皆僅獲得0.93的準確度，比羅吉斯分類器少很多，這是已經嘗試多種參數後獲得的最佳結果。而在這兩筆資料為二差別是，pca的資料花費較短的時間變跑完，且在weighted avg時，由原始資料略勝一籌（多0.1）。而這個分類器總體運作速度比羅吉斯快很多。雖然準確度不夠高，但速度卻能達到。
3. 在神經網路中，我將原始資料與pca資料選用相同的激活函數logistic，相同的隱藏曾神經元數量120，不同的演算法，一個是lbfgs，一個是sgd。其中以pca資料的準確度較高，他擁有95%的準確度，而原始資料則表現不特別突出，只有92%，是所有之中最低的。而這個分類器相比svm花的時間也較長。

建議

1. 如果希望準確度越高越好，那最適合讓標準化原始資料在羅吉斯分類器底下進行分類，並選用liblinear演算法。
2. 如果希望能有不錯的準確度，同時效能高一點，不要花費太久的時間，資料也不會失真的情況下，可以在神經網路底下用95%主成分的資料，並採用 hidden_layers = (120,) ; activation = 'logistic' ; solver = 'sgd'的參數設

可能導致準確度很低的原因

- 資料特性不適合模型：
  - ex: 資料具有線性可分離的特性-->羅吉斯迴歸可能會表現更好；
  - ex: 資料複雜度高-->複雜的神經網路
- 資料預處理：
  - 標準化-->幫助模型更快收斂
  - 高維度資料-->PCA降維可能會丟失重要信息，
- 模型參數調整
- 資料量和質量：
  - 資料量太少
  - 質量差，模型可能受到噪音、雜訊影響